PETSAPP

Support Engineer — Take-Home Task

Please complete before your Stage 2 interview

Before you start — please read this

This task is designed to take 3-4 hours. Please do not spend more than that — we are not looking for perfection.

We want to see how you think, how you communicate, and how you approach problems practically.

You may use any tools you like, including AI tools (LLMs, Copilot, etc.). If you do, briefly note where and how — this is not a penalty, it is part of what we want to understand.

Submit your work as a single PDF/GitHub repo link within 7 days of receiving the task.

Context

PetsApp integrates with veterinary clinic management systems (called PMS — Practice Management Systems). When a clinic books an appointment or updates a patient record in their PMS, our system syncs that data so we can send reminders, messages, and notifications to pet owners.

Syncs can fail. Data can arrive in unexpected formats. Clinics get confused. Your job as a Support Engineer is to figure out what went wrong, fix it where you can, communicate clearly, and reduce the chance of it happening again.

The three tasks below are based on real scenarios from this role.

The Tasks

Task 1 Diagnose a Broken Sync

A veterinary clinic called Paws & Claws has raised a support ticket. Their appointments are not syncing into PetsApp — reminder messages are not going out to owners. They are frustrated because this worked fine last week.You have been given the following log excerpt from our sync service (simplified for this task):

What to submit:

- A written diagnosis: what do you think is wrong, and why?
- The questions you would ask the clinic (in plain, empathetic language — write this as if it is a real message to them)
- The steps you would take internally to investigate and resolve it
- If you were writing a Jira bug ticket based on this, what would it look like?

```
// Sync log — Paws & Claws — 2026-04-14 09:02 UTC
```

```
[INFO]  Starting sync for group_id: 4821
[INFO]  Fetching appointments from PMS API...
[INFO]  PMS response received: 200 OK
[INFO]  Parsing appointment records...
[WARN]  Unexpected field format on record 003: "appointmentDate" received "14/04/2026",
expected ISO 8601
[WARN]  Unexpected field format on record 004: "appointmentDate" received "14/04/2026",
expected ISO 8601
[ERROR] Validation failed: 2 of 6 records rejected (invalid date format)
[INFO]  Sync completed with partial success: 4 records written, 2 skipped
[INFO]  Downstream notification skipped: partial sync flagged, full sync required before
reminders fire
```

Task 2 Write a TypeScript Utility Function

One of the root causes of the issue above (and many like it) is that our system receives dates in different formats depending on which PMS a clinic uses. Some send ISO 8601 ("2026-04-14T09:00:00Z"), some send DD/MM/YYYY, some send MM-DD-YYYY, and occasionally a clinic will have misconfigured their PMS and send something unparseable.Write a TypeScript function that normalises incoming date strings into a consistent ISO 8601 UTC timestamp.

What to submit:

- A TypeScript function: normaliseAppointmentDate(raw: string): string | null
- It should handle at minimum: ISO 8601, DD/MM/YYYY, and MM-DD-YYYY
- Return null (with a comment explaining why) for inputs that cannot be parsed
- A few lines of comments explaining your reasoning
- A short set of test cases (Jest or plain assertions — whatever you prefer)

Tip

Don't overthink the date parsing library choice — use whatever you are comfortable with (date-fns, dayjs, Luxon, or native Date). Briefly note why you chose it.

We are not looking for a perfect production-ready implementation — we want to see how you reason about edge cases and write clear, maintainable code.

Task 3 Reduce Future Ticket Volume

Imagine you have now resolved the Paws & Claws issue and investigated further — you discover that 6 other clinics using the same PMS (let's call it VetCore) have the same date format misconfiguration. It has caused intermittent sync failures across all of them for the past two weeks.You want to: (a) communicate with affected clinics, and (b) make sure this does not happen again.

What to submit:

- A short customer-facing message to send to the 6 affected clinics explaining what happened and what to do (if anything) — in plain English.
- A brief internal proposal (a few bullet points or a short paragraph) for how to prevent this class of issue recurring.
- If you were to use an AI tool to help with any part of this — the investigation, the communication, the fix — where would you use it and how?

Submission

Please send your submission to this email within 7 days of receiving the task. You can submit:

- A single PDF document covering tasks 1 and 3
- A GitHub repo (even a private one — just add us as a collaborator) with a README for task 2.